Assignment: Tomasulo Algorithm with Multiple Issue

COSC 6385 Advanced Computer Architecture
Pradham Mummaleti

Result: the simulator now issues 1 to 4 instructions per cycle (in order). All 4 of my test cases end with the same registers and memory at every issue width, and the test with independent instructions goes from **27 cycles at width 1 to 19 cycles at width 4**. With issue width 1 the timing table for the given code.in is exactly the same as the original code.

1. What I added to the code

I started from the given tomasulo-python code. It already had the ROB, RAT, reservation stations, the load/store queue and one CDB, but it could only issue one instruction per cycle. These are the changes I made (each one is marked in the code with a comment that says added by me, changed by me or fixed by me):

Issue width input (main.py). The issue width can be given as a line in the input file, like *Issue width = 2*, or on the command line, for example *python3 main.py test.in 2*. It is limited to 1 to 4 and the default is 1. I also made main.py read register and memory init lines from the input file (like *R1=10, F2=2.5* and *Mem[8]=5.0*) and the number of ROB entries, so each test can set up its own values.

Issuing more than one instruction (issue.py). I renamed the original issue function to issue_one and wrote a new issue function that calls it up to issue_width times in the same cycle:

```
def issue(..., issue_width=1):
    count = 0
    while count < issue_width:
        if PC.PC >= len(instructions) or PC.valid == 0:
            break
        old_rob_len = len(ROB)
        issue_one(...)
        if len(ROB) == old_rob_len:
            break      # could not issue (stall), stay in order
        count += 1
```

It stops early in two cases. If the next instruction cannot issue because its reservation station, the ROB or the load/store queue is full, nothing behind it can issue either, since issue has to stay in order. And after a Bne, PC.valid is 0 until the branch resolves, so issue stops there too. The RAT is updated after every single instruction, so if two dependent instructions issue in the same cycle the second one still gets the first one's ROB tag instead of the old register value.

More results per cycle (wb.py and commit.py). If you issue 4 per cycle but can only broadcast and commit 1 per cycle, the ROB just fills up. So I changed the CDB into a list of buses and broadcast up to issue_width results per cycle, and the commit stage now runs the original commit logic up to issue_width times per cycle. With width 1 both behave exactly like before.

Output (main.py). At the end it prints the total number of cycles, the register values and the non-zero memory values, in the format from the handout.

2. Bugs I found and fixed while testing

When I ran my own test cases I got some wrong answers, even with issue width 1, so I tracked them down and fixed them:

(a) RAT overwritten by an older instruction. In test 2, R3 is written twice. When the first write broadcast, the code put its value into the RAT even though the RAT already pointed at the newer instruction, so the next instruction read the old R3 and got R5 = 107 instead of 111. Now the RAT is only updated if it still holds that instruction's tag (wb.py).

(b) Last store lost. The main loop stopped as soon as the ROB was empty, but a store leaves the ROB when it starts writing memory, which takes 5 cycles. If the store was the last instruction, the value never got written (test 3 was missing Mem[20] = 24). The loop now keeps going while the memory unit or the load/store queue is still busy (main.py).

(c) Pipelined units moving too fast. When a new instruction started on the FP adder, the code called the pipeline function again, which moved every instruction already in the pipeline forward a second time, so some FP adds took 3 cycles instead of 4. Now a new instruction is started on its own (exe.py).

(d) Two instructions starting on one unit in the same cycle. This only showed up with multi issue: two loads computed their address in the same cycle even though there is only one address adder, and two Addi instructions used the single integer adder in the same cycle. Each unit can now start at most one instruction per cycle, and it picks the oldest ready instruction (by ROB order) instead of whatever is in the lowest reservation station slot (exe.py).

After all of these fixes I checked that the timing table for the original code.in with issue width 1 is still exactly the same as the output of the original code.

3. My test cases

The test files are in tests/. I worked out the correct final values by hand for each one so I could check correctness, not just speed.

Test	What it checks	Expected
test1_independent	10 ALU ops with no dependencies	R4=11 R5=22 R6=30 R7=15 R8=8 R9=10, F3=4.0 F4=1.0 F5=3.0 F6=3.75
test2_dependent	RAW chain, plus WAR/WAW on R3, issued in the same cycle	R3=11 R4=14 R5=111 R6=25
test3_memory	loads, stores, store-to-load forwarding	F4=12.0 F5=12.0 F6=24.0, Mem[16]=12.0 Mem[20]=24.0
test4_loop_branch	the given code.in (loop with Bne)	same values as the original code

4. Results under different issue widths

Test	Width 1	Width 2	Width 3	Width 4	Same final state?
test1_independent	27	22	20	19	yes
test2_dependent	11	11	11	11	yes
test3_memory	43	43	43	43	yes
test4_loop_branch	39	36	36	36	yes

Total cycles for each test (all output files are in results/)

For every test the registers and memory at the end are the same for widths 1 to 4, and they match the values I worked out by hand. Below is test 1 at width 1 and at width 4. At width 4 the first four instructions issue in cycle 1 and the next four in cycle 2:

Test 1, issue width 1

Instruction	Issue	EX	MEM	WB	Commit
Addi R4 R1 1	1	2-2	-	3	4
Addi R5 R2 2	2	3-3	-	4	5
Add R6 R1 R2	3	4-4	-	5	6
Sub R7 R2 R3	4	5-5	-	6	7
Add.d F3 F1 F2	5	6-9	-	10	11
Sub.d F4 F2 F1	6	7-10	-	12	13
Add.d F5 F1 F1	7	8-11	-	13	14
Mult.d F6 F1 F2	8	9-23	-	24	25
Addi R8 R3 3	9	10-10	-	11	26
Add R9 R3 R3	10	11-11	-	14	27

Test 1, issue width 4

Instruction	Issue	EX	MEM	WB	Commit
Addi R4 R1 1	1	2-2	-	3	4
Addi R5 R2 2	1	3-3	-	4	5
Add R6 R1 R2	1	4-4	-	5	6
Sub R7 R2 R3	1	5-5	-	6	7
Add.d F3 F1 F2	2	3-6	-	7	8
Sub.d F4 F2 F1	2	4-7	-	8	9
Add.d F5 F1 F1	2	5-8	-	9	10
Mult.d F6 F1 F2	2	3-17	-	18	19
Addi R8 R3 3	3	6-6	-	7	19
Add R9 R3 R3	4	7-7	-	8	19

Test 2, issue width 4 (all four dependent instructions issue in cycle 1 and still get the right values)

Instruction	Issue	EX	MEM	WB	Commit
Add R3 R1 R2	1	2-2	-	3	4
Add R4 R3 R3	1	4-4	-	5	6
Sub R3 R4 R1	1	6-6	-	7	8
Addi R5 R3 100	1	8-8	-	9	10
Add R6 R3 R4	2	9-9	-	10	11

5. What I noticed

Multi issue helps the most when the instructions are independent. Test 1 drops from 27 to 19 cycles, about 30 percent. Going from width 3 to width 4 only saves one more cycle, because there is only one of each functional unit and the integer adder can only start one instruction per cycle, so the instructions issue faster but then wait for the unit.

Test 2 stays at 11 cycles at every width because every instruction needs the one before it, so wider issue cannot help a dependence chain. Test 3 stays at 43 cycles at every width because the memory is single

ported and takes 5 cycles per access, and the loads, the add and the stores mostly wait on each other through memory. Test 4 goes from 39 to 36 and then stays there, since there is no branch prediction: issue stops at every Bne until it resolves, so after width 2 there is nothing left to issue in the gap, and the last store to memory finishes at the same time anyway.

6. What works and what does not

Works: issue width 1 to 4 (in order), correct renaming when dependent instructions issue together, up to issue_width broadcasts and commits per cycle, register and memory output, and all four test cases at every width. Width 1 gives the same timing as the original code.

Does not / limitations: the given Python code does not have the 1-bit branch predictor or the BTB, so the simulator still stalls issue at every Bne until it resolves, and I did not add prediction. Beq is not supported (the original only has Bne). The number of reservation stations, functional unit latencies and memory latency are still set in main.py instead of being read from the input file table. Load and store addresses only work with single digit base registers like 0(R1), which is how the original code parses them. I assumed the number of CDBs and the commit width are the same as the issue width.

References

Assignment handout (Tomasulo Algorithm Implementation) and the provided tomasulo-python code.